

RELATÓRIO DE PROJETO
PROGRAMAÇÃO ORIENTADA A OBJETOS

JAVA CAFE - POINT SALE SYSTEM

Mateus Henrique Demetrio Ferreira - 17074492

Mateus Augusto Silva de Carvalho - 17074450

Andrew Brancalhona Moreira - 6254193

1. Requisitos

O trabalho pedia a implementação de uma aplicação GUI em Java Swing, que representasse um ponto de venda de uma cafeteria, e trazia uma série de requisitos obrigatórios a serem seguidos, os quais são listados a seguir:

Primeiramente, a aplicação deveria se basear em uma interface gráfica de usuário interativa, limpa e intuitiva que se dividisse em várias telas: uma tela para realizar pedidos, uma para gerenciar o inventário da loja, e uma que gerasse um relatório de vendas para diferentes períodos.

A interface de usuário deveria se basear em botões de menu que permitissem a entrada rápida de pedidos, de forma simples e intuitiva, tanto para selecionar itens do menu, como para realizar ou cancelar o pedido, e como para realizar operações de inventário e interagir com o relatório de vendas. Além disso, deveria expor constantemente o total do pedido, mantendo-o sempre atualizado.

Outro requisito era permitir a seleção de pedidos em várias quantidades, calculando automaticamente um subtotal, um imposto e um total do pedido, gerando um recibo em texto que pudesse ser impresso ou salvo ao final do pedido.

Em termos de estoque, os requisitos eram manter a lista de produtos, contando com campos de nome, preço e quantidade em estoque. A lista deveria ser dinâmica, de forma que, a cada pedido, a quantidade de itens no estoque fosse reduzida automaticamente, permitindo que empregados pudessem adicionar novos produtos e atualizar produtos já existentes, e permitindo ainda configurar um limite de itens no estoque para que, caso um estoque caísse abaixo desse limite, o programa exibirá um aviso.

Quanto ao relatório de vendas, eram requisitos poder visualizar relatórios referentes ao dia, à semana e ao mês atual, com cada relatório mostrando a receita total do período, o número de transações e os três itens mais vendidos nesse período.

Ainda, era requisito que os registros de estoque e vendas fossem salvos em arquivos locais, configurando a aplicação para carregar os dados na inicialização do sistema a partir dos dados gravados nos arquivos, permitindo que o estado atual persistisse entre diferentes inicializações.

Por fim, o último requisito era o tratamento de exceções personalizadas como `OutOfStockException` (Produto Esgotado) e `InvalidPaymentException` (Pagamento Inválido), e mensagens de erro amigáveis e claras para o usuário.

2. Descrição do projeto

O projeto foi feito para atender aos requisitos mencionados acima, e portanto, consiste em uma aplicação GUI usando Swing, formada em uma única janela com três abas, uma para cada tela de pedidos, inventário e relatório de vendas. A aplicação obtém os dados de estoque e vendas na inicialização a partir de arquivos .csv, bem como atualiza esses arquivos enquanto roda, sempre que alguma alteração é feita através da interface gráfica, e portanto, torna-se dinâmica, mantendo seu estado sempre atualizado.

A interface de pedidos, projetada para ser intuitiva, permite a seleção rápida de itens do cardápio, como cafês, tortas e cappuccinos, através de botões visuais ilustrados. O cálculo do subtotal, impostos e valor total é atualizado em tempo real no menu lateral, à medida que os itens são adicionados ou removidos. A finalização da venda gera um cupom fiscal formatado em texto, que pode ser salvo localmente como arquivo .txt ou impresso, garantindo praticidade e registro adequado.

O módulo de controle de inventário proporciona monitoramento dinâmico através de uma tabela (JTable) que exibe o nome, preço e quantidade em estoque de cada produto. O estoque é atualizado em tempo real, com a quantidade de produtos decrementada automaticamente a cada venda finalizada. A edição do inventário é flexível, permitindo que funcionários ajustem valores diretamente na tabela (com validações de formato) ou adicionem novos produtos através de um formulário. O sistema também inclui um controle crítico de estoque, com alertas visuais automáticos caso o nível de estoque fique abaixo de um limite configurável, garantindo que a cafeteria nunca fique sem os ingredientes essenciais.

O painel de vendas (Sales Dashboard) apresenta um dashboard modernizado que exibe indicadores de performance financeira (KPIs) segmentados por períodos: Hoje, Semana Atual e Mês Atual. As métricas apresentadas incluem a receita total calculada, a quantidade de transações concluídas e o top 3 produtos mais vendidos, destacados com badges visuais dinâmicos. A exportação de relatórios está disponível, permitindo a extração de dados do dashboard em formatos de texto formatado (.txt) ou planilha (.csv) para análises mais aprofundadas.

Conforme dito acima, a persistência de dados é garantida através do armazenamento local, com todas as transações financeiras salvas em sales.csv e o estoque atualizado mantido em inventory.csv. Os dados são recarregados automaticamente ao abrir a aplicação, garantindo a continuidade das operações. Para assegurar a consistência e evitar problemas, o sistema inclui tratamento de exceções customizadas, como a `OutOfStockException`, que

impede a venda de itens indisponíveis no estoque e gera avisos visuais amigáveis, auxiliando os funcionários na gestão do inventário

3. Comentários acerca do código

Aqui está o texto adaptado para o Português Brasileiro, com as correções de vocabulário (como trocar "ficheiro" por "arquivo", "ecrã" por "tela", "stock" por "estoque", entre outras adaptações gramaticais):

Da parte das vendas e atendimento, o código também está dividido entre a interface visual (OrderGUI) e a lógica de processamento (OrderLogic). Na OrderGUI, nós construímos a interface do ponto de venda (POS) utilizando o Java Swing. A área central é composta por um menu dinâmico que gera botões com as imagens dos produtos disponíveis no inventário. Adicionamos um recurso de segurança visual: se a imagem de um produto não for encontrada localmente, o nosso código gera automaticamente um bloco cinza de substituição com o texto "No Img", evitando falhas e garantindo a correta apresentação gráfica. À direita, implementamos um carrinho de compras através de uma JTable não editável, que lista os itens selecionados, as quantidades e os preços, além de apresentar os cálculos do subtotal, taxas e valor total.

Na classe OrderLogic, nós programamos todo o "motor" por trás da interface de vendas. Através da implementação da interface ActionListener, conseguimos captar os cliques nos botões dinâmicos do menu. Quando um produto é selecionado, o nosso código se comunica dinamicamente com o inventário para obter o preço atualizado e adiciona o item à tabela do carrinho. Se o item já existir no carrinho, a quantidade é apenas incrementada, recalculando de imediato o subtotal, a taxa fixa de 10% e o valor total da compra. De igual forma, desenvolvemos métodos precisos para remover itens específicos (reduzindo a quantidade gradualmente ou removendo a linha) e para cancelar o pedido por completo, repondo os valores a zero de forma limpa.

O grande destaque desta seção é o processo de finalização da compra no método de submissão do pedido. Antes de finalizar, o nosso código realiza uma verificação em tempo real, consultando a lógica de inventário para garantir que há estoque suficiente para cada item, apresentando um erro caso algum produto exceda a quantidade disponível. Se a validação for bem-sucedida, a venda é salva no histórico, o estoque é deduzido automaticamente e os relatórios são atualizados. Por fim, nós implementamos um sistema que gera um recibo formatado em texto, detalhando a data, hora e os itens transacionados.

Através de uma caixa de opções, damos ao usuário a possibilidade de imprimir o recibo fisicamente utilizando a impressora do sistema ou salvá-lo em um arquivo de texto (.txt) com um nome único gerado com base na data da operação.

Da parte do estoque, o código também é separado entre a parte visual (InventoryGUI) e a lógica (InventoryLogic). Esses códigos compõem a parte de gerenciamento do estoque, onde estruturamos todas as funções de cadastro, controle e persistência dos nossos produtos. Na InventoryGUI, nós utilizamos a biblioteca Java Swing para criar uma interface interativa e fácil de manusear. Nela, implementamos uma tabela dinâmica (JTable) equipada com validações que impedem a inserção de preços ou estoques negativos. Um dos recursos mais interessantes que nós desenvolvemos nessa tela foi o sistema de imagens: configuramos um seletor de arquivos (JFileChooser) que aceita apenas formatos válidos (como JPG e PNG). Quando uma foto é selecionada no computador, o nosso código faz uma cópia automática desse arquivo para a pasta /imgs do projeto, garantindo que o sistema sempre encontre as imagens corretamente sem o risco de links quebrados.

Já na InventoryLogic, nós concentramos todo o 'cérebro' dessas operações. Construímos métodos para que os dados do inventário sejam lidos e salvos de forma persistente em um arquivo de texto (inventory.csv). Além de criarmos um sistema que gera alertas automáticos quando o estoque de um item fica baixo, o grande diferencial que nós implementamos foi a sincronização em tempo real. Nós configuramos listeners que monitoram a tabela de estoque; assim, toda vez que cadastramos um produto novo, o nosso sistema avisa a aba de pedidos (OrderGUI) para recriar o menu dinamicamente. Dessa forma, conseguimos fazer com que a vitrine de vendas reflita o nosso banco de dados instantaneamente, sem precisar reiniciar o programa.

Para a gestão e registro das vendas, nós desenvolvemos a classe SalesPersistence, responsável por garantir que o histórico das transações seja armazenado de forma segura e permanente. Utilizamos as bibliotecas nativas de manipulação de arquivos do Java (como BufferedReader e PrintWriter) para ler e gravar as informações de forma eficiente em um arquivo local com o formato CSV (sales.csv). Estruturamos o código criando classes internas para representar cada item vendido e a transação completa, registrando o momento exato de cada operação através da classe LocalDateTime. Adicionalmente, implementamos uma função de segurança que detecta se o arquivo está vazio na primeira execução, gerando automaticamente dados simulados (mock data) para que o sistema possa ser testado e exibido de imediato.

Na componente visual, nós criamos a `SalesReportGUI`, um dashboard analítico moderno e atraente construído em Java Swing. Focamos fortemente na estética e na experiência do usuário, motivo pelo qual desenvolvemos componentes gráficos personalizados, tais como cartões com cantos arredondados e sombras dinâmicas (`ShadowCard`) e botões planos com efeitos de hover (`FlatButton`). Esta tela exibe em tempo real as métricas mais importantes do negócio: a receita total acumulada, o número de transações efetuadas e um pódio calculado dinamicamente com os três artigos mais vendidos. Como a classe implementa `ActionListener`, todas as interações respondem de forma imediata sem bloquear o programa.

O grande diferencial tecnológico deste módulo é o sistema avançado de filtragem e exportação que nós programamos. Utilizando a API de datas moderna do Java (`TemporalAdjusters`), adicionamos um filtro dropdown que permite ao usuário alternar instantaneamente a visualização das métricas entre o desempenho de "Hoje", da "Semana Atual" ou do "Mês Atual". Para complementar a análise, incluímos uma funcionalidade de exportação através do `JFileChooser`. O nosso código lê o estado atual do filtro e compila um relatório estruturado que o usuário pode escolher salvar num arquivo de texto limpo e formatado (.txt) ou como uma planilha (.csv), facilitando a contabilidade externa do café.

4. Plano dos testes

Para assegurar a confiabilidade do sistema, foram implementados testes automatizados que validam suas funções críticas. O teste de persistência de vendas (**testSalesPersistence**) tem como objetivo garantir que a aplicação consiga gravar e ler o histórico de transações corretamente. Durante sua execução, é criada uma lista de itens simulados — incluindo três cafés *Espresso* a \$2.50 e um *Muffin* a \$4.00 — acionando em seguida os métodos de salvamento e leitura em um arquivo CSV. As validações confirmam se exatamente uma transação foi salva, se o valor total calculado corresponde ao montante esperado de \$11.50 e se os nomes e as quantidades dos itens carregados são idênticos aos que foram originalmente gravados no arquivo.

Na verificação das métricas e filtros do painel, o teste de interface de relatórios (**testSalesReportGUI**) atesta se o sistema consegue ler o banco de dados de vendas e exibir as informações financeiras adequadas. O procedimento apaga o histórico existente e força a injeção de uma venda de duas tortas (*Pies*), totalizando \$9.00, com a data do dia corrente. Após instanciar a tela de relatórios, o teste simula a seleção do filtro "Hoje". Através da

técnica de *Reflection* para acessar os campos privados da interface, o sistema assegura que a tela exiba corretamente a receita de \$9.00 e contabilize exatamente uma transação concluída.

Por fim, o teste de gerenciamento de pedidos (**testOrderLogic**) avalia o fluxo completo de adição e cancelamento de produtos no carrinho. Para evitar a dependência de dados reais do usuário, é injetado um inventário fictício contendo uma torta a \$4.50 e um café a \$2.50. Conectando a lógica de pedidos à de inventário, o sistema simula cliques nos botões de compra desses produtos. As validações garantem que a tabela do carrinho seja atualizada progressivamente para uma e depois duas linhas, e que o valor total exibido na interface seja de \$7.70, refletindo com exatidão a soma dos produtos acrescida da taxa de 10% de imposto. O teste também simula o clique no botão de cancelar o pedido, certificando-se de que a tabela do carrinho é totalmente esvaziada e o valor de cobrança retorna a zero.

5. Resultados dos testes

A execução automatizada validou a integridade de todas as lógicas testadas. O sistema processou cálculos matemáticos de carrinho, tributação (taxas), leitura/escrita em arquivos .csv, e as atualizações na tabela de interface (DefaultTableModel) sem apresentar falhas ou exceções estruturais.

Todos os critérios de aceitação estipulados pelos *Asserts* (como o cálculo exato do total de R\$ 7,70 com a taxa) foram atendidos com sucesso.

O output dos testes foi o seguinte:

```
=====
RUNNING JAVA CAFE UNIT TESTS
=====
Running test: SalesPersistence Save and Load ... PASSED
Running test: SalesReportGUI Filtering and Metrics ... PASSED
Running test: OrderLogic Item Management ... PASSED
Running test: InventoryLogic Operations ... PASSED
=====
Test Summary: 4/4 tests passed.
=====
```

6. Guia de como rodar o código

Para rodar esse código, são necessários como softwares básicos um Java Development Kit (JDK), em versão 17 ou superior, e um terminal de sistema.

Tendo esses requisitos, basta clonar o repositório através do terminal com “git clone <https://github.com/mat-hdf/Java-Cafe---Point-Sale-System.git>”, e entrar nesse diretório com o comando “cd Java-Cafe---Point-Sale-System”.

Feito isso, compile o código com “javac *.java” e inicie a aplicação principal Java Cafe com “java Main”.

7. Problemas

Dentre alguns problemas encontrados ao longo do desenvolvimento do projeto, podem ser mencionados alguns, como:

A dificuldade de salvamento de estado, importando dados de um arquivo e depois exportando-os novamente, tornando a lógica de instanciamento do programa mais complexa.

Outra dificuldade encontrada foi a integração de códigos criados por diferentes membros do grupo, alinhando as ideias e arquiteturas que cada um criou e combinando-as em uma única estrutura íntegra, funcional e coesa.

Por fim, podemos mencionar a dificuldade em criar uma UI intuitiva, de fácil entendimento, acesso, e coesa, que reunisse todas as informações necessárias em um único lugar e se mantivesse acessível a qualquer usuário, fosse ele leigo ou não.

No entanto, acredita-se que todos esses problemas foram enfrentados e superados, chegando a um resultado final satisfatório para o presente projeto.

8. Comentários Adicionais

O projeto final apresenta uma interface interativa, simples, intuitiva e estilizada. O projeto cumpre com todos os requisitos obrigatórios, e ainda conta com o cumprimento de um dos requisitos adicionais mencionados, isto é, a exportação do relatório de vendas para um arquivo de texto.

Além disso, a GUI foi estilizada, para garantir uma aparência agradável ao usuário, pensando em sua experiência usando a aplicação. Com isso, todas as funcionalidades se tornam simples de usar, atrativas, e concentram-se todas em um único lugar, garantindo eficiência no design do projeto.

Com isso, os retoques finais do projeto ficam concluídos, resultando em uma interface funcional, simples e agradável.